Introduction to Big Data Assignment 2 Report

Student: Aisylu Fattakhova
Email: a.fattakhova@innopolis.university
GitHub Repository: https://github.com/AisyluFattakhova/Big-Data-Assignment-2

---

1. Methodology

1.1 Overview

In this assignment, I implemented a simple search engine using Hadoop MapReduce, Spark, and Cassandra. The system consists of three main components:

- Data preparation (PySpark)
- Indexing (Hadoop MapReduce)
- Storage (Cassandra)
- Ranking (Spark BM25)

1.2 Data Preparation

The source data consists of Wikipedia articles from a Kaggle dataset.

1. **Extraction:** 100 documents were extracted and saved as individual `.txt` files.
2. **Standardization:** Using PySpark, files were processed into a Tab-Separated format: `doc_id [TAB] doc_title [TAB] doc_text`.
3. **HDFS Storage:** The formatted data was uploaded to HDFS at `/input/data`. This format allows the MapReduce Mappers to easily split the input by tabs.

1.3 Indexing (MapReduce Pipelines)

I implemented two distinct MapReduce pipelines to generate the components needed for BM25.

Pipeline 1: Inverted Index

- **Mapper (`mapper1.py`):** Tokenizes the text, removes non-alphabetic characters using regex, converts to lowercase, and filters out common stopwords. It emits `word \t doc_id_title`.
- **Reducer (`reducer1.py`):** Aggregates word counts across documents. It produces a compressed postings list in the format: `word \t doc1:count|doc2:count`.

Pipeline 2: Document Statistics

- **Mapper (`mapper2.py`):** Identifies the length (total word count) of each document.
- **Reducer (`reducer2.py`):** Finalizes the document stats. This is essential for calculating the average document length ($avgdl$) used in BM25 normalization.

1.4 Storage in Cassandra

The MapReduce outputs were migrated from HDFS to Cassandra using `app.py`. The schema was designed for high-speed lookups:

- `inverted_index` : Maps words to their postings.
- `vocabulary` : Stores the document frequency (df) for each word.
- `doc_stats` : Stores document lengths (dl).

1.5 Ranking Algorithm (BM25)

The ranking is performed in `query.py` using Spark. The BM25 score for a document D and query Q is calculated as:

Inverse Document Frequency (IDF):

$$\mathrm{IDF}(q) = \log\left(1 + \frac{N - df + 0.5}{df + 0.5}\right)$$

BM25 Score:

$$\mathrm{Score}(D, Q) = \sum_{q \in Q} \mathrm{IDF}(q) \cdot \frac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\mathrm{avgdl}}\right)}$$

Where:

- $f(q, D)$ is the term frequency of word q in document D.
- $|D|$ is the length of document D.
- $avgdl$ is the average document length in the corpus.
- $k_1 = 1.5$ and $b = 0.75$.

1.6 Design Choices

- **Cleaning Logic:** I used regex `[^a-zA-Z\s]` to remove noise (numbers/symbols), ensuring the index contains only meaningful words.
- **Postings Format:** Using a pipe-separated string (`doc:count|doc:count`) reduces the number of rows in Cassandra, making term retrieval more efficient than storing one row per word-document pair.
- **Spark Partitioning:** I used `foreachPartition` in the loading script to minimize the overhead of opening/closing Cassandra connections.

2. Demonstration

2.1 How to Run

The repository is fully containerized. To execute the entire pipeline:

1. *(Optional)* Place the `a.parquet` file into the `app/` folder.

- If provided, the system will extract and process up to 100 documents automatically.
- If not provided, the system will still work using the pre-existing `.txt` files located in the `app/data/` directory.

1. Run:
```
docker compose up --build
```
2. The system will automatically start Hadoop, index 100 documents, load Cassandra, and run a sample query for "history".
3. To run a custom query manually:
```
docker exec -it cluster-master bash /app/search.sh "your_query"
```

2.2 Results and Screenshots

HDFS Directory Structure

```
root@cluster-master: /app
user@DESKTOP-PQTU5K7:~$ docker exec -it cluster-master bash
root@cluster-master:/app# hdfs dfs -ls /input/data
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-04-04 18:19 /input/data/_SUCCESS
-rw-r--r--   1 root supergroup    3521946 2026-04-04 18:19 /input/data/part-00000
root@cluster-master:/app# hdfs dfs -ls /indexer/index
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-04-04 18:19 /indexer/index/_SUCCESS
-rw-r--r--   1 root supergroup    7630792 2026-04-04 18:19 /indexer/index/part-00000
root@cluster-master:/app# hdfs dfs -ls /indexer/stats
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-04-04 18:20 /indexer/stats/_SUCCESS
-rw-r--r--   1 root supergroup      12770 2026-04-04 18:20 /indexer/stats/part-00000
root@cluster-master:/app#
```

The successful creation of output files in HDFS. Both `/indexer/index` and `/indexer/stats` contain `_SUCCESS` flags and `part-00000` files, confirming the MapReduce jobs finished correctly.

Cassandra Tables

```
user@DESKTOP-PQTU5K7: ~
user@DESKTOP-PQTU5K7:~$ docker exec -it cassandra-server cqlsh
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> USE search_engine;
cqlsh:search_engine> SELECT * FROM inverted_index LIMIT 5;

 word        | postings
-------------+-----------------------------------------------------------------------------------
      dhands |                                                                   54187039_A_A_Dhand:2
  laoghaires |                                                 3307869_A_Breath_of_Snow_and_Ashes:1
        sain |                                                          28784108_A_Black_Man's_Soul:1
       await | 89397_A_Bridge_Too_Far_(film):1|4136530_A_Bay_of_Blood:1|89541_A_Better_Tomorrow:1
 ccclxxxviii |                                                 35717559_A_Beuk_o'_Newcassell_Sangs:1

(5 rows)
cqlsh:search_engine> SELECT * FROM doc_stats LIMIT 10;

 doc_id   | doc_length
----------+------------
 33456414 |        688
 38294693 |        360
 68147671 |        253
  1719074 |       1613
 61110799 |        206
   504808 |       1339
   195945 |       4429
  8100298 |       1237
 45349228 |        177
  2301932 |       1446

(10 rows)
cqlsh:search_segine> SELECT * FROM vocabulary LIMIT 10;

 word        | doc_count
-------------+-----------
      dhands |         1
  laoghaires |         1
        sain |         1
       await |         3
 ccclxxxviii |         1
  discograph |         1
       dance |        33
    previews |         6
       gibbs |         6
   courtneys |         1

(10 rows)
cqlsh:search_engine>
```

Verification of the data inside Cassandra using `cqlsh`. It shows the `inverted_index` with word postings, `doc_stats` with document IDs and lengths, and `vocabulary` with word document counts. This proves the Spark migration script correctly populated the NoSQL layer.

Raw MapReduce Outputs (Inverted Index)

```
root@cluster-master: /app
root@cluster-master:/app# hdfs dfs -cat /input/data/part-00000 | head -n 2
10078432        A_Case_for_the_Court    A Case for the Court was a weekly CBC Television show that ran from July 1960 to September 1962.CBC/film archive at queensu.ca The show was produced in cooperation with th
e Canadian Bar Association, involving the enactment of fictional criminal and civil cases using actual judges and lawyers. ==References== ==External links== * *A Case for the Court - Canadian Communication Found
ation Category:1960 Canadian television series debuts Category:1962 Canadian television series endings Category:CBC Television original programming
10119231        A_Balladeer     A Balladeer (styled as a balladeer) is a Dutch band, originating from Amsterdam, founded by singer-songwriter Marinus de Goederen. In 2002, the band won the 3FM BuZz Award and ano
ther award the following year at the final of the annual Dutch musical contest De Grote Prijs van Nederland (The Big Prize of the Netherlands). In 2004, A Balladeer opened up for international acts Saybia and Ke
ane and released their first EP: Rumor Had It. The EP featured 4 songs, including the critically acclaimed "They've Shut Down Marks & Spencer", which resulted in considerable airplay in the Netherlands. On 5 May
 2006, A Balladeer's debut album titled Panama was released by EMI Music. The single "Swim with Sam" was the first single. It became a hit in the Netherlands. The album was recorded in Brussels in 2005 with Bløf
 producer Ronald Vanhuffel. Robert Kirby (known for his work with Nick Drake) arranged the strings for three songs. In early 2007, the band won a Zilveren Harp (Silver Harp), a supporting prize for promising mus
ical artists. In February 2007, their third single was released, a ballad called "Robin II". The second album, Where Are You, Bambi Woods?, was released on 29 August 2008. The album is partly about the year the
lead singer lived near Dallas, Texas. Its title track is about the disappearance of porn star Bambi Woods (known from her role in Debbie Does Dallas). 'Poster Child' is an ode to the murdered student Matthew She
pard. In 2009 the album got nominated for an Edison, a Dutch Grammy. The third album, I Can't Keep Track Of Each Fallen Robin, was released on 19 October 2012. The single 'Mob Wife' (2017) features Sam Bettens o
f K's Choice. The original version of the song is taken from A Wolf at the Door, which was released on 11 March 2016. == Band line up == === Current members === * Marinus de Goederen - vocals, piano, guitar ===
Past members === * Walter Wilhelm - bass guitar (2003-2004) * Erik Meereboer - guitar, backing vocals (2003-2010) * Tijs Stehmann - drums (2003-2013) === Session members === * Peter Slager (Bløf) - bass guitar (
studio) * Bas Kennis (Bløf) - Hammond, accordion (studio) * Charlie Dée - backing vocals * Arn Kortooms - bass guitar, vocals (live) == Discography == === Albums === * Panama (2006) * Where Are You, Bambi Woods?
 (2008) * I Can't Keep Track of Each Fallen Robin (2012) * A Wolf at the Door (2016) * December (2017) * Clutter, Volume 1 (2020) === EPs === * Rumor Had It (2004) === Singles === * "Swim with Sam" (2006) * "For
tune Teller" (2006) * "Robin II" (2007) * "Mary Had A Secret" (2008) * "Superman Can't Move His Legs" (2008) * "Oh, California" (2010) * "Karaoke Night" (2012) * "A Wolf At The Door" (2016) * "Incompatible" (201
6) * "Mob Wife (Ft. Sam Bettens)" (2017) * "Not Only During Christmas" (2017) ==External links== *Official website Category:Musical groups from Amsterdam
cat: Unable to write to output stream.
root@cluster-master:/app# hdfs dfs -cat /indexer/index/part-00000 | head -n 20
aaa     47880532_A_Aa:2|35235985_A_Buddy_Story:1|2828410_A_(musical_note):1
aaaa    2828410_A_(musical_note):1
aabccb  316851_A_Boy_Named_Sue:1
aaca    9424803_A_Breed_of_Heroes:3
aahh    70140038_A_Camel's_Kiss:1
aakeson 40383589_A_Caretaker's_Tale:1
aal     53349884_A_Boy_and_a_Girl_with_a_Cat_and_an_Eel:2
aalborg 61515396_A_25-Year_Celebration_Tour:1
aaliyah 3033793_A_Change_Is_Gonna_Come:1
aaltonen        73939988_A_Charming_Mass_Suicide_(film):1
aame    22811539_A_Aa_E_Ee_(2009_Telugu_film):1
aanand  47880532_A_Aa:2
aanands 47880532_A_Aa:1
aanwezige       59959284_A_Cabinet_of_Curiosities_(painting):1
aap     52178053_A_Boogie_wit_da_Hoodie:1|10423764_A_Bathing_Ape:2
aape    10423764_A_Bathing_Ape:1
aarno   73939988_A_Charming_Mass_Suicide_(film):1
aaron   785456_A_Big_Hunk_o'_Love:5|58435794_A_Beautiful_Now:1|51864384_A_Can_of_Paint:1|63582800_A_Certain_Doom:1|62779553_A_Beautiful_Place_to_Drown:2|51244890_A_Billion_Lives:2|29012311_A_Beautiful_Day:1
aarp    56450073_A_Beautiful_Day_in_the_Neighborhood:1|47134700_A_Bigger_Splash_(2015_film):1|54641828_A_Banquet_for_Hungry_Ghosts:1
aarseth 2054290_A_Blaze_in_the_Northern_Sky:1
cat: Unable to write to output stream.
root@cluster-master:/app#
```

A `cat` command on the HDFS part files. The first part shows the formatted TSV document data. The second part shows the resulting inverted index where words are mapped to specific documents and their frequencies.

Document Count and Stats

```
root@cluster-master: /app
root@cluster-master:/app# hdfs dfs -cat /indexer/stats/part-00000 | head -n 20
10078432        74
10119231        500
10196530        241
10223157        192
10245514        181
10258008        109
10260791        577
10355999        198
10423764        821
10486281        197
10534798        389
1053640 824
10559911        106
10660999        45
10746498        203
10957294        693
10966581        760
11017293        459
11022705        438
11066006        3765
root@cluster-master:/app# hdfs dfs -cat /input/data/part-00000 | wc -l
1000
root@cluster-master:/app#
```

This screenshot shows the document stats (ID vs Length) and a `wc -l` command confirming that 1000 documents were successfully processed and indexed into the system.

Search Engine Execution

```
root@cluster-master: /app
        start time: 1775385428561
        final status: UNDEFINED
        tracking URL: http://cluster-master:8088/proxy/application_1775325487447_0005/
        user: root
26/04/05 10:37:13 INFO YarnClientSchedulerBackend: Add WebUI Filter. org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter, Map(PROXY_HOSTS -> cluster-master, PROXY_URI_BASES -> http://cluster-master:8088/p
roxy/application_1775325487447_0005), /proxy/application_1775325487447_0005
26/04/05 10:37:13 INFO YarnSchedulerBackend$YarnSchedulerEndpoint: ApplicationMaster registered as NettyRpcEndpointRef(spark-client://YarnAM)
26/04/05 10:37:13 INFO Client: Application report for application_1775325487447_0005 (state: RUNNING)
26/04/05 10:37:13 INFO Client:
        client token: N/A
        diagnostics: N/A
        ApplicationMaster host: 172.18.0.3
        ApplicationMaster RPC port: -1
        queue: default
        start time: 1775385428561
        final status: UNDEFINED
        tracking URL: http://cluster-master:8088/proxy/application_1775325487447_0005/
        user: root
26/04/05 10:37:13 INFO YarnClientSchedulerBackend: Application application_1775325487447_0005 has started running.
26/04/05 10:37:13 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 44371.
26/04/05 10:37:13 INFO NettyBlockTransferService: Server created on cluster-master:44371
26/04/05 10:37:13 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/04/05 10:37:13 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, cluster-master, 44371, None)
26/04/05 10:37:13 INFO BlockManagerMasterEndpoint: Registering block manager cluster-master:44371 with 366.3 MiB RAM, BlockManagerId(driver, cluster-master, 44371, None)
26/04/05 10:37:13 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, cluster-master, 44371, None)
26/04/05 10:37:13 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, cluster-master, 44371, None)
26/04/05 10:37:18 INFO YarnSchedulerBackend$YarnDriverEndpoint: Registered executor NettyRpcEndpointRef(spark-client://Executor) (172.18.0.3:38858) with ID 1,  ResourceProfileId 0
26/04/05 10:37:18 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:46287 with 366.3 MiB RAM, BlockManagerId(1, cluster-slave-1, 46287, None)
26/04/05 10:37:20 INFO YarnSchedulerBackend$YarnDriverEndpoint: Registered executor NettyRpcEndpointRef(spark-client://Executor) (172.18.0.3:38872) with ID 2,  ResourceProfileId 0
26/04/05 10:37:20 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:45681 with 366.3 MiB RAM, BlockManagerId(2, cluster-slave-1, 45681, None)
26/04/05 10:37:20 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.8

================================================================================
BM25 TOP 10 RESULTS FOR: history
================================================================================
Rank  | Doc ID     | Document Title                                        | Score
--------------------------------------------------------------------------------
1     | 60121915   | A Brief History of Everyone Who Ever Lived            | 3.7187
2     | 44671392   | A Brief History of Chinese Fiction                    | 3.6869
3     | 67227      | A Brief History of Time                               | 3.6869
4     | 23727098   | A Brief History of Love                               | 3.5552
5     | 1443000    | A Brief History of Time (film)                        | 3.5552
6     | 3579086    | A & C Black                                           | 3.4158
7     | 1636834    | A Century of Dishonor                                 | 3.3182
8     | 34353837   | A Brief History of Blasphemy                          | 3.3182
9     | 67215164   | A Boy with a Flying Squirrel                          | 3.3182
10    | 24667353   | A Brief History of Anxiety (Yours & Mine)             | 3.1906
================================================================================

root@cluster-master:/app#
```

```
root@cluster-master: /app
        start time: 1775385500532
        final status: UNDEFINED
        tracking URL: http://cluster-master:8088/proxy/application_1775325487447_0006/
        user: root
26/04/05 10:38:24 INFO YarnClientSchedulerBackend: Add WebUI Filter. org.apache.hadoop.yarn.server.webproxy.amfilter.AmIpFilter, Map(PROXY_HOSTS -> cluster-master, PROXY_URI_BASES -> http://cluster-master:8088/p
roxy/application_1775325487447_0006), /proxy/application_1775325487447_0006
26/04/05 10:38:24 INFO YarnSchedulerBackend$YarnSchedulerEndpoint: ApplicationMaster registered as NettyRpcEndpointRef(spark-client://YarnAM)
26/04/05 10:38:24 INFO Client: Application report for application_1775325487447_0006 (state: RUNNING)
26/04/05 10:38:24 INFO Client:
        client token: N/A
        diagnostics: N/A
        ApplicationMaster host: 172.18.0.3
        ApplicationMaster RPC port: -1
        queue: default
        start time: 1775385500532
        final status: UNDEFINED
        tracking URL: http://cluster-master:8088/proxy/application_1775325487447_0006/
        user: root
26/04/05 10:38:24 INFO YarnClientSchedulerBackend: Application application_1775325487447_0006 has started running.
26/04/05 10:38:24 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 37713.
26/04/05 10:38:24 INFO NettyBlockTransferService: Server created on cluster-master:37713
26/04/05 10:38:24 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
26/04/05 10:38:24 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, cluster-master, 37713, None)
26/04/05 10:38:24 INFO BlockManagerMasterEndpoint: Registering block manager cluster-master:37713 with 366.3 MiB RAM, BlockManagerId(driver, cluster-master, 37713, None)
26/04/05 10:38:24 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, cluster-master, 37713, None)
26/04/05 10:38:24 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, cluster-master, 37713, None)
26/04/05 10:38:28 INFO YarnSchedulerBackend$YarnDriverEndpoint: Registered executor NettyRpcEndpointRef(spark-client://Executor) (172.18.0.3:44420) with ID 1,  ResourceProfileId 0
26/04/05 10:38:28 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:35813 with 366.3 MiB RAM, BlockManagerId(1, cluster-slave-1, 35813, None)
26/04/05 10:38:29 INFO YarnSchedulerBackend$YarnDriverEndpoint: Registered executor NettyRpcEndpointRef(spark-client://Executor) (172.18.0.3:44432) with ID 2,  ResourceProfileId 0
26/04/05 10:38:29 INFO BlockManagerMasterEndpoint: Registering block manager cluster-slave-1:41385 with 366.3 MiB RAM, BlockManagerId(2, cluster-slave-1, 41385, None)
26/04/05 10:38:29 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached minRegisteredResourcesRatio: 0.8

================================================================================
BM25 TOP 10 RESULTS FOR: spring
================================================================================
Rank  | Doc ID     | Document Title                                        | Score
--------------------------------------------------------------------------------
1     | 7214270    | A Case of Spring Fever                                | 7.3447
2     | 63462998   | A Ballad About Green Wood                             | 6.2318
3     | 3401447    | A Ceremony of Carols                                  | 5.7125
4     | 36253472   | A Bite of China                                       | 5.7125
5     | 67215164   | A Boy with a Flying Squirrel                          | 4.8964
6     | 72355109   | A Blonde Woman                                        | 4.8964
7     | 460442     | A Bug's Life                                          | 4.8964
8     | 59892611   | A Cambodian Spring                                    | 4.8964
9     | 4089573    | A Buyer's Market                                      | 4.8964
10    | 38370970   | A (Jimmy Raney album)                                 | 4.8964
================================================================================

root@cluster-master:/app#
```

Execution of a query. The engine retrieves the relevant documents from Cassandra, calculates the BM25 score in Spark, and displays the Top 10 results with their Rank, Doc ID, Title, and Score.

---

3. Reflections and Findings

- Using MapReduce for indexing allows the system to process files in parallel. While processing 1000 documents is fast, the same logic scales to millions of documents.
- The BM25 algorithm successfully handled document length normalization. For example, a shorter document with 3 occurrences of a word often ranked higher than a very long document with 5 occurrences, which is the expected behavior of a robust search engine.
- Integrating Cassandra provided a significant performance boost for queries compared to scanning HDFS files directly. The bottleneck shifted from disk I/O to network latency between Spark and Cassandra.